

System Design Report

Point-of-Sale (POS) System

CSC-225 Software Engineering

1 Introduction

This document presents the system design for the Point-of-Sale (POS) System developed for a pizza shop, as part of Project Milestone 3 of the course CSC-225 (Software Engineering). The objective of this milestone is to translate the approved Software Requirements Specification (SRS) into a complete and consistent system design.

The design focuses on visualizing the structure, behavior, and interactions of the system using standard software engineering modeling techniques. The system design strictly follows the functional and non-functional requirements defined in the SRS and adheres to IEEE 830 guidelines. Key design artifacts include use case diagrams, data flow diagrams, sequence diagrams, activity diagrams, class diagrams, and component diagrams.

This design ensures traceability between requirements and design, validates system behavior before implementation, and reduces risks related to requirement ambiguity, inconsistency, and incomplete coverage.

2 Design Assumptions and Constraints

2.1 Design Assumptions

The following assumptions were made during the system design phase:

- All users (Waiter, Cashier, Kitchen Staff, and Admin) possess basic knowledge of POS systems and touchscreen interfaces.
- The POS system operates within a restaurant environment where all terminals are connected via a Local Area Network (LAN).
- Payment gateways for online transactions are provided by reliable third-party services and expose standard APIs.
- Required hardware devices such as desktops, tablets, receipt printers, and cash drawers are available and functioning correctly.

- The system is used during defined restaurant operating hours and supports concurrent access by multiple users.

2.2 Design Constraints

The following constraints influenced the system design:

- The system must comply with the requirements and structure defined in the approved SRS document.
- The design must adhere to IEEE 830 standards for software documentation.
- The system must operate on Windows 10 or newer and Android 10 or newer platforms.
- Performance constraints require real-time order updates with minimal latency across all system interfaces.
- Security constraints require role-based access control and secure payment processing using encrypted communication.
- The system must support at least 50 concurrent users and up to 500 transactions per day without performance degradation.

These assumptions and constraints guided architectural choices, component decomposition, and interaction design throughout the system.

3 Key Design Decisions

This section highlights important design decisions made during the system design phase and provides justification for each decision.

3.1 Process Decomposition in DFDs

The system was decomposed hierarchically in Data Flow Diagrams (DFDs), starting from a Level 0 context diagram and expanding to Level 1 and Level 2. Order Management, Kitchen Processing, Billing and Payment, and Administration were identified as separate processes to improve clarity, scalability, and maintainability.

3.2 Order Lifecycle Management

A state-based order lifecycle (Order Pending → In Preparation → Prepared → Served → Completed) was adopted to ensure consistent order tracking across waiter, kitchen, and cashier interfaces. This decision simplifies synchronization and prevents invalid state transitions.

3.3 Separation of Responsibilities

The system design enforces a clear separation of responsibilities by assigning distinct roles to system actors. This decision supports security, minimizes unauthorized access, and ensures that each user interacts only with features relevant to their role.

3.4 Class Relationships

Associations were used between core classes such as *Order*, *MenuItem*, and *Payment* to represent real-world relationships. Aggregation was preferred over composition where entities can exist independently, such as MenuItems existing independently of Orders.

3.5 Component-Based Architecture

A component-based architecture was selected to separate major functional modules including Order Management, Kitchen Module, Billing Module, Admin Module, and User Interface. This design decision improves modularity, simplifies testing, and supports future system extensions without major redesign.

3.6 Support for Multiple Payment Methods

Billing and payment processing were designed to support cash, card, and online payment methods. This decision was made to enhance system flexibility and align with modern restaurant payment practices.

Overall, these design decisions ensure that the system design is robust, scalable, secure, and fully traceable to the documented requirements.